

A view of 0 or 1 elements: `view::maybe`

- Document number: P1255R0
- Date: 2018-10-07
- Author: Steve Downey <sdowney2@bloomberg.net>
- Audience: LEWG

Abstract: This paper proposes `view::maybe` a range adaptor that produces a view with cardinality 0 or 1 which acts as an adaptor for nullable types such as `std::optional` and pointer types.

Table of Contents

- [1. Motivation](#)
- [2. Proposal](#)
- [3. Design](#)
- [4. Very very preliminary wording](#)
- [5. Impact on the standard](#)
- [6. Bikeshed](#)
- [7. References](#)

1 Motivation

In writing range transformation pipelines it is useful to be able to lift a nullable value into a view that is either empty or contains the value held by the nullable. The adapter `view::single` fills a similar purpose for non-nullable values, lifting a single value into a view, and `view::empty` provides a range of no values of a given type. A `view::maybe` adaptor also allows nullable values to be treated as ranges when it is otherwise undesirable to make them containers, for example `std::optional`.

```
std::vector<std::optional<int>> v{
    std::optional<int>{42},
    std::optional<int>{},
    std::optional<int>{6 * 9}};

auto r = view::join(view::transform(v, view::maybe));

for (auto i : r) {
    std::cout << i; // prints 42 and 42
}
```

In addition to range transformation pipelines, `view::maybe` can be used in range based for loops, allowing the nullable value to not be dereferenced within the body. This is of small value in small examples in contrast to testing the nullable in an if statement, but with longer bodies the dereference is often far away from the test. Often the first line in the body of the `if` is naming the dereferenced nullable, and lifting the dereference into the for loop eliminates some boilerplate code, the same way that range based for loops do.

```
{
    auto&& opt = possible_value();
    if (*opt) {
        // a few dozen lines ...
        use(*opt); // is *opt OK
    }
}
```

```

}

// vs ...

for (auto&& v : view::maybe(possible_value())) {
    // a few dozen lines ...
    use(v);
}

```

2 Proposal

Add class template `maybe_view`, a closure object `view::maybe`, and customization point objects `view::maybe_has_value` and `view::maybe_value`, used to determine if there is a value in the underlying object and to produce a `View` that contains that value.

3 Design

The basis of the design is to hybridize `view::single` and `view::empty`. If the underlying object holds a value, as determined by `maybe_has_value` on the object, `begin` and `end` of the view are equivalent to `return maybe_value(value_).operator->();`, and `return begin()+1`. Otherwise, if the underlying object does not have a value, `begin` and `end` return `nullptr`.

The names `has_value` and `value` are chosen to mirror the member functions of `std::optional`, and can pass through to those. The proposed `std::expected` uses the same names, establishing a pattern for nullable types. If the type being ranged over is pointer-like then `maybe_has_value` is equivalent to `!= nullptr`, and `maybe_value` dereferences the underlying object.

4 Very very preliminary wording

4.1 Class template `maybe_view`

`maybe_view` produces a `View` that contains exactly zero or one element contained within a specified nullable value.

[Example:

```

maybe_view s{std::optional{4}};
for (int i : s)
    cout << i; // prints 4
maybe_view e{std::optional{}};
for (int i : e)
    cout << i; // does not print

```

— end example]

```

namespace std::ranges {
    template<CopyConstructible T>
    requires is_object_v<T>
    class maybe_view : public view_interface<maybe_view<T>> {
    private:
        T* value_; // exposition only
    public:

```

```

maybe_view() = default;

template<typename U>
requires Constructible<T, remove_pointer<U>>>
constexpr explicit maybe_view(const U& u);

constexpr T* begin() const noexcept;

constexpr T* end() const noexcept;

constexpr static ptrdiff_t size() noexcept;

constexpr const T* data() const noexcept;
};
template<class U>
explicit maybe_view(const U&) -> maybe_view<invoke_result<decltype(maybe_value), U>>;
}

```

4.2 maybe_view operations

```

template<typename U>
constexpr explicit maybe_view(const U& u);

```

Effects: Initializes value_ with addressof(maybe_value(u)).

```

constexpr T* begin() const noexcept;

```

Effects: Equivalent to: return value_;

```

constexpr T* end() const noexcept;

```

Effects: Equivalent to: if (value_ != nullptr) return value_.operator->() + 1; else return nullptr;

```

constexpr static ptrdiff_t size() noexcept;

```

Effects: Equivalent to return !value_;

```

constexpr const T* data() const noexcept;

```

Effects: Equivalent to: return begin();

4.3 view::maybe

The name view::maybe denotes a customization point object ([customization.point.object]). The expression view::maybe(E) for some subexpression E is expression-equivalent to maybe_view{E}.

4.4 view::maybe_has_value

The name `view::maybe_has_value` denotes a customization point object ([customization.point.object]). The expression `view::maybe_has_value(E)` for some subexpression `E` is expression-equivalent to `E.has_value()` or `(E != nullptr)` if `E.has_value()` is not well formed.

[Example:

```
constexpr std::optional s{7};
constexpr std::optional<int> e{};
static_assert(view::maybe_has_value(s));
static_assert(!view::maybe_has_value(e));
```

— end example]

4.5 `view::maybe_value`

The name `view::maybe_value` denotes a customization point object ([customization.point.object]). The expression `view::maybe_value(E)` for some subexpression `E` is expression-equivalent to `*(E.operator->())`.

[Example:

```
std::optional s{42};
assert(view::maybe_value(s) == 42);

int k = 42;
int *p = &k;
assert(view::maybe_value(p) == 42);
```

— end example]

5 Impact on the standard

Dependent on The One Ranges Proposal, P0896, but otherwise a pure library extension.

6 Bikeshed

The name `maybe`.

That `maybe_view` is templated only on the held type of the nullable and not the nullable type. This allows mixing of nullable types in the conversion to `view`, at the cost of not being able to recover the base.

7 References

[P0896R2] Eric Niebler, Casey Carter, Christopher Di Bella. The One Ranges Proposal URL: <https://wg21.link/p0896r2>

[P0323R7] Vicente Botet, JF Bastien. `std::expected` URL: <https://wg21.link/p0323r7>